Architecture des processeurs et optimisation

November 2017 ***Documents allowed - Duration 2h***

An unjustified answer will be considered as false

It is imperative to use the provided squared sheets for the simplified diagrams

Consider the P7 pipeline processor. This processor has the same instruction set as the **Mips**. It is built around a 7-stage pipeline : **IF1, IF2, DEC, EXE, MEM1, MEM2** and **WBK**.

The pipeline was obtained from the pipeline of the **Mips** implementation. The **IFC** and **MEM** stages of the Mips pipeline were each divided into two stages. The next instruction address is calculated in the **EXE** stage. The implementation contains all the bypasses required to support data dependencies. However, there is no bypass at the input of the **MEM1** stage.

On this implementation, we wish to evaluate the execution performance of a function that calculates the distribution of pixel values in an image (known as the *histogram* application), for pixels with a value below a threshold. The C code of the *histogram* function is as follows :

```
void histogram (unsigned char *img, int size, int histo [256],
                unsigned char  threshold)
  {
  for (int i = 0; i < size; i += 1)
    {
    if (img[i] < hreshold)  histo [img[i]] += 1;
    }
  }
```

A **non-pipelined** Mips compiler has produced two versions for the code corresponding to the function's loop. In accordance with *gcc* conventions, at function's input, register r4 contains the value of **img** (image start address), register r5 contains the **size** parameter (array size), register r6 contains the start address of the **histo** array and register r7 contains the value of the **threshold** parameter. Register r10 is initialized with the address of the last element of **img.**

Version 1 :

```
    _loop:   Lbu    r8 , 0 (r4 )              # read img[i]
             Sltu   r9 , r8 , r7              # img[i] < threshold ?
             Beq    r9 , r0 , _endif
             Sll    r8 , r8 , 2
             Addu   r9 , r6 , r8              # address of histo[img[i]]
             Lw     r11, 0 (r9 )              # read       histo[img[i]]
             Addiu  r11, r11, 1
             Sw     r11, 0 (r9 )              # update     histo[img[i]]
    _endif:  Addiu  r4 , r4 , 1
             Bne    r4 , r10, _loop
```

Version 2 :

```
    _loop:   Lbu    r8 , 0 (r4 )              # read img[i]
             Sltu   r9 , r8 , r7              # img[i] < threshold ?
             Sll    r8 , r8 , 2
             Addu   r11, r6 , r8              # address of histo[img[i]]
             Lw     r12, 0 (r11)              # read       histo[img[i]]
             Addu   r12, r12, r9
             Sw     r12, 0 (r11)              # update     histo[img[i]]
             Addiu  r4 , r4 , 1
             Bne    r4 , r10, _loop
```

Throughout the exercise, we assume that in the **`img`** array, the probability that $img[i] < threshold$ is 70%.

1- Does the organization of the pipeline in 7 stages have any impact on the compiler ? Please explain.

2- In the **Mips** processor, there are data dependencies of order 1, 2 and 3. In **P7**, to what order do data dependencies extend ? Justify your answer with a simplified diagram.

3- What bypasses are required to execute instructions in this **P7** ? Illustrate the use of each bypass with a simplified diagram and an example of a sequence of instructions.

4- For **Version 1**, modify the code so that it becomes executable on the **P7** realization. Analyze the execution of this loop using a simplified diagram, assuming that the branch fails. Calculate the average number of cycles per pixel. Calculate the CPI and CPI-useful.

5- For **Version 2**, modify the code so that it becomes executable on the **P7** realization. Analyze the execution of this loop using a simplified diagram. Calculate the average number of cycles per pixel.

6- For each of the two versions, propose an optimized code by modifying the instruction order and by using speculative execution. You don't need to draw a diagram, but simply indicate the freeze cycles on the optimized code. For each of the two versions, calculate the average number of cycles per pixel.

7- For each of the two versions, unroll the code once (processing 2 pixels at each iteration) and optimize (by changing the instruction order and using speculative execution). You don't need to draw a diagram, but simply indicate the stall cycles on the code after optimization. For each of the two versions, calculate the average number of cycles per pixel.

8- We would like to compare execution on the **P7** with execution on the **Mips**. For the original version of each of the two versions, calculate the average number of cycles per element for execution on the **Mips**. You don't need to draw a diagram, but simply indicate the freeze cycles on the code.

   For each of the two versions, what must be the minimum ratio between the frequencies of the **P7** and the **Mips** for execution to be more performant on the **P7** ?